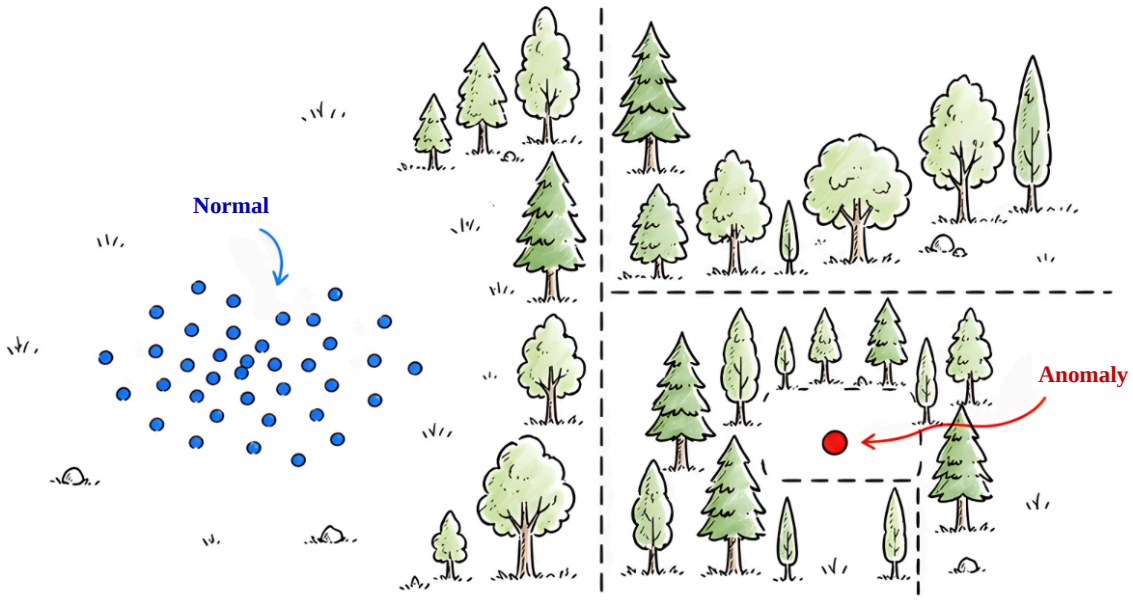
# Isolation Forest for Anomaly Detection

Trần Hoàng Duy  Đinh Quang Vinh

## I. Giới thiệu

Trong nhiều hệ thống thực tế, dữ liệu không phải lúc nào cũng tuân theo những mẫu hành vi quen thuộc. Bên cạnh phần lớn các quan sát bình thường, đôi khi sẽ xuất hiện một vài điểm dữ liệu khác biệt rõ rệt và cần được chú ý. Bài toán phát hiện những điểm như vậy được gọi là *anomaly detection*.




Hình 1: **Minh họa điểm bất thường trong dữ liệu.** Phần lớn điểm dữ liệu nằm gần nhau tạo thành nhóm bình thường, trong khi điểm nằm tách biệt được xem là điểm bất thường.

Trong hình minh họa, phần lớn dữ liệu nằm gần nhau và tạo thành nhóm bình thường, trong khi điểm màu đỏ nằm tách biệt hơn so với các điểm còn lại. Đây là trực quan cơ bản của anomaly detection: tìm ra những quan sát không giống với xu hướng chung của dữ liệu.

Bài viết này giới thiệu **Isolation Forest**, một phương pháp phát hiện bất thường dựa trên ý tưởng cô lập dữ liệu. Thay vì mô hình hóa toàn bộ phân bố dữ liệu bình thường, phương pháp này kiểm tra xem một điểm có thể bị tách riêng nhanh đến mức nào bằng các phép chia ngẫu nhiên. Từ đó, ta có thể đánh giá điểm nào có khả năng là anomaly.

# Mục lục

**I. Giới thiệu** . . . . . **1**

**II. Anomaly Detection** . . . . . **3**

1. Anomaly Detection là gì? . . . . . 3
2. Các dạng anomaly thường gặp . . . . . 4
3. Ý tưởng dẫn đến Isolation Forest . . . . . 5

**III. Isolation Forest** . . . . . **6**

1. Isolation Tree . . . . . 6
2. Path Length . . . . . 7
3. Từ một cây đến một Forest . . . . . 7
4. Anomaly Score . . . . . 9

**IV. Thực hành** . . . . . **10**

1. Bộ dữ liệu . . . . . 10
2. Output trong Scikit-learn . . . . . 10
3. Pipeline tổng quát . . . . . 11
4. Cài đặt chương trình . . . . . 12
5. Đánh giá mô hình . . . . . 14

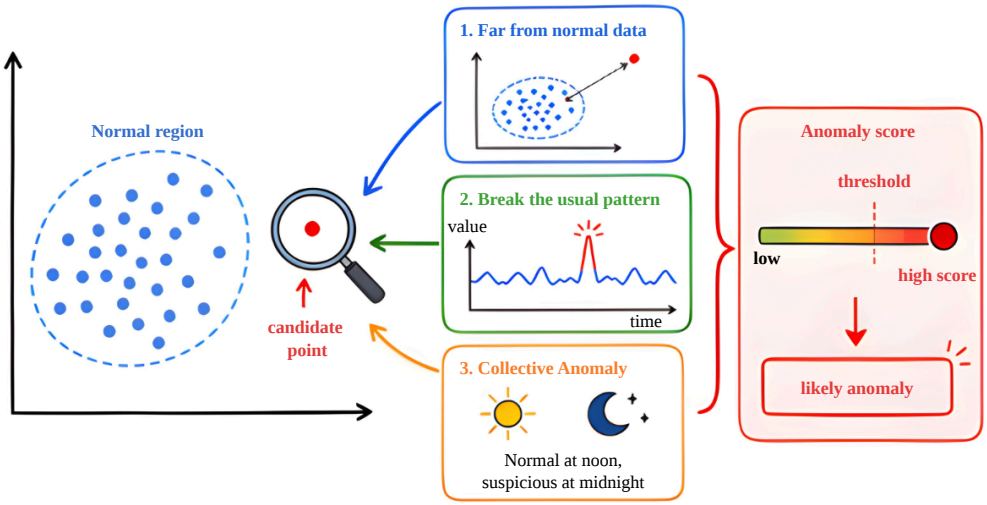
**V. Tài liệu tham khảo** . . . . . **15**

**Phụ lục** . . . . . **15**

# II. Anomaly Detection

## 1. Anomaly Detection là gì?

Anomaly detection (phát hiện bất thường) là bài toán tìm ra những điểm dữ liệu khác biệt đáng kể so với phần lớn dữ liệu còn lại. Trong thực tế, những điểm này thường gắn với các tình huống quan trọng như giao dịch gian lận, lỗi thiết bị, truy cập bất thường, dữ liệu cảm biến bị hỏng, hay các hành vi hiếm gặp trong hệ thống.




Hình 2: **Minh họa cách phát hiện điểm bất thường.** Một điểm được xem là bất thường khi nằm xa vùng dữ liệu bình thường, phá vỡ quy luật quen thuộc hoặc xuất hiện trong ngữ cảnh không phù hợp.

Điều quan trọng cần nhấn mạnh là một anomaly không nhất thiết đồng nghĩa với lỗi. Nó chỉ cho biết rằng điểm dữ liệu đó không giống với số đông và vì vậy đáng để kiểm tra thêm. Chẳng hạn, một giao dịch chuyển tiền giá trị rất lớn có thể là gian lận, nhưng cũng có thể chỉ là một giao dịch hợp lệ hiếm gặp. Vì vậy, anomaly detection thường được xem như bước sàng lọc và cảnh báo sớm, chứ không phải lúc nào cũng là bước đưa ra kết luận cuối cùng.

Bài toán anomaly detection khó hơn nhiều so với vẻ ngoài của nó. Lý do đầu tiên là anomaly thường rất hiếm. Nếu chỉ có một tỉ lệ rất nhỏ dữ liệu là bất thường, mô hình rất dễ bị thiên lệch về phía dữ liệu bình thường. Lý do thứ hai là trong nhiều hệ thống thực tế, ta không có nhãn rõ ràng cho từng mẫu. Nói cách khác, ta thường không biết trước mẫu nào là normal và mẫu nào là anomaly.

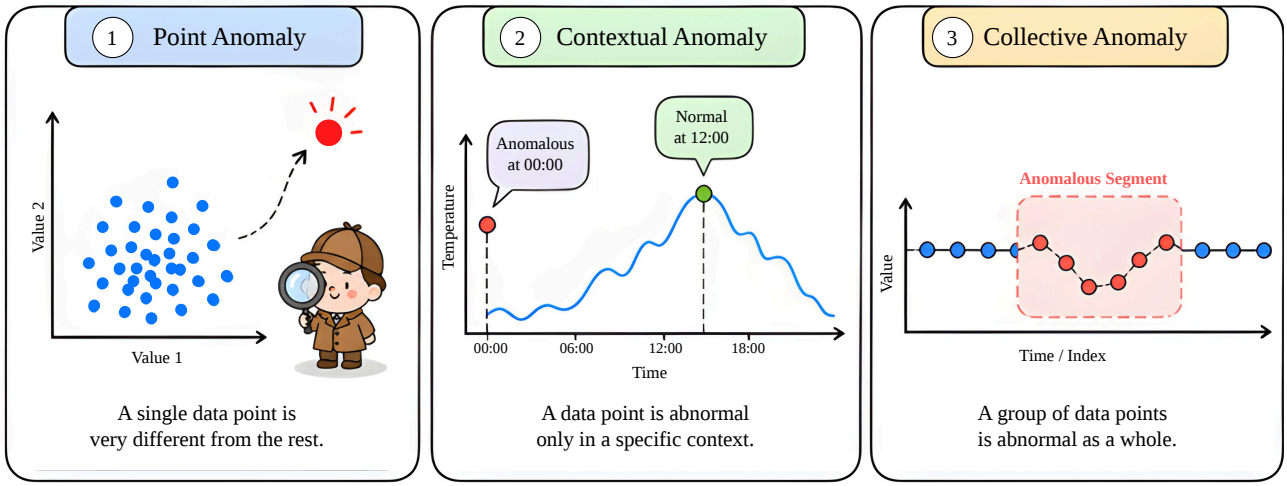
Ngoài ra, khái niệm "bất thường" còn phụ thuộc mạnh vào ngữ cảnh. Một giá trị có thể bình thường trong hoàn cảnh này nhưng lại bất thường trong hoàn cảnh khác. Khi dữ liệu có nhiều chiều, bài toán còn khó hơn vì một điểm có thể trông bình thường nếu xét từng feature riêng lẻ, nhưng lại bất thường khi xét tổ hợp nhiều feature cùng lúc.

**Điểm cần nhớ**

Anomaly detection không chỉ là tìm "điểm lạ", mà là tìm những quan sát đáng được chú ý vì chúng khác số đông hoặc khác với ngữ cảnh kỳ vọng.

## 2. Các dạng anomaly thường gặp

Trong thực tế, không phải mọi bất thường đều giống nhau. Tùy vào bản chất dữ liệu và ngữ cảnh, ta thường gặp ba dạng anomaly phổ biến: point anomaly, contextual anomaly và collective anomaly.




Hình 3: **Ba dạng điểm bất thường thường gặp.** Anomaly có thể xuất hiện dưới dạng một điểm khác biệt, một điểm bất thường trong ngữ cảnh cụ thể, hoặc một nhóm điểm bất thường khi xét chung với nhau.

### 2.1 Point anomaly

Point anomaly là dạng trực quan nhất. Một điểm dữ liệu đơn lẻ được xem là bất thường nếu nó khác biệt rõ rệt so với phần lớn dữ liệu còn lại. Ví dụ, nếu phần lớn nhiệt độ vận hành của một máy nằm trong khoảng 60–70°C, thì một quan sát ở mức 110°C có thể được xem là point anomaly.

### 2.2 Contextual anomaly

Contextual anomaly là trường hợp một điểm dữ liệu chỉ trở nên bất thường khi xét trong đúng ngữ cảnh của nó. Bản thân giá trị chưa đủ để kết luận. Ta cần xét thêm các yếu tố như thời gian, mùa vụ, vị trí hoặc trạng thái hệ thống. Ví dụ, doanh số tăng mạnh trong dịp lễ có thể là bình thường, nhưng mức tăng tương tự vào một ngày bình thường có thể là tín hiệu đáng chú ý.

### 2.3 Collective anomaly

Collective anomaly xảy ra khi một nhóm điểm dữ liệu cùng nhau tạo thành hành vi bất thường, mặc dù từng điểm riêng lẻ có thể không quá đặc biệt. Ví dụ, một chuỗi request nhỏ gửi liên tục trong thời gian ngắn có thể là tín hiệu của tấn công hoặc lỗi hệ thống, dù mỗi request đơn lẻ vẫn trông hợp lệ.

**Điểm cần nhớ**

Point anomaly tập trung vào một điểm đơn lẻ. Contextual anomaly phụ thuộc vào ngữ cảnh. Collective anomaly phụ thuộc vào hành vi của một nhóm điểm.

## 3. Ý tưởng dẫn đến Isolation Forest

Sau khi hiểu anomaly detection là bài toán tìm những điểm khác biệt, câu hỏi tiếp theo là:

> *Làm thế nào để biết được một điểm có bất thường hay không?*

Một số phương pháp anomaly detection cố gắng học vùng dữ liệu bình thường trước, sau đó kiểm tra xem điểm nào nằm quá xa hoặc không phù hợp với vùng đó. Cách tiếp cận này hợp lý, nhưng trong thực tế, vùng dữ liệu bình thường có thể rất phức tạp và khó mô hình hóa chính xác.

Isolation Forest đi theo một hướng khác. Thay vì cố gắng học đầy đủ vùng dữ liệu bình thường, nó đặt ra một câu hỏi đơn giản hơn:

> *Một điểm dữ liệu có dễ bị cô lập khỏi phần còn lại hay không?*

Cách hiểu trực quan ở đây là các điểm bất thường thường nằm ở vùng thưa dữ liệu, ở rìa phân bố, hoặc nằm xa phần lớn các điểm còn lại. Vì vậy, nếu ta liên tục chia dữ liệu bằng các phép chia ngẫu nhiên, các điểm này thường bị tách riêng nhanh hơn so với các điểm bình thường nằm trong vùng đông đúc.

Nói cách khác, nếu một điểm chỉ cần vài lần chia ngẫu nhiên là đã đứng một mình, điểm đó có khả năng là anomaly. Ngược lại, nếu một điểm cần nhiều lần chia mới có thể được cô lập, nó có xu hướng là normal.

**Điểm cần nhớ**

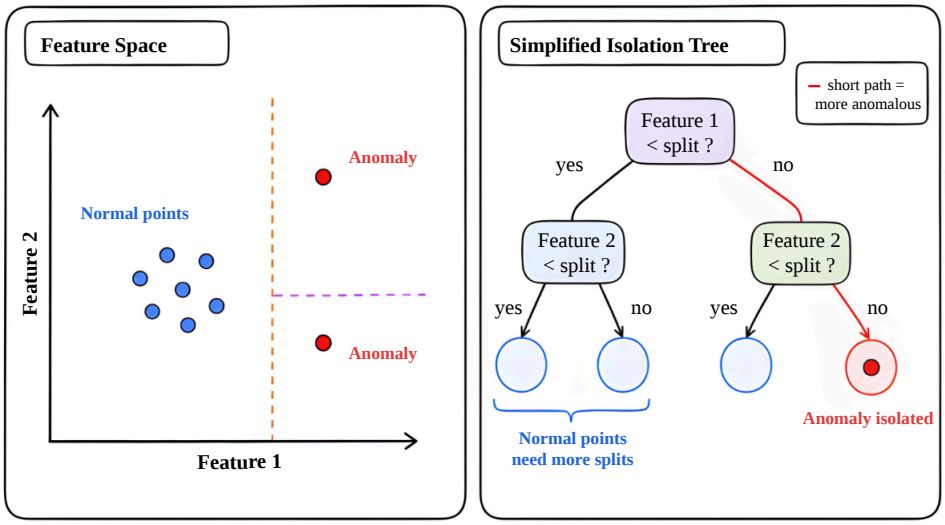
Điểm càng dễ bị cô lập khỏi phần còn lại thì điểm đó càng có khả năng là anomaly.

Từ trực quan này, ta đi đến thuật toán **Isolation Forest**, một phương pháp phát hiện bất thường dựa trên việc xây dựng nhiều cây ngẫu nhiên để đo độ dễ bị cô lập của từng điểm dữ liệu.

# III. Isolation Forest

## 1. Isolation Tree

Thành phần cơ bản của Isolation Forest là *Isolation Tree*, thường gọi tắt là *iTree*. Có thể hiểu *iTree* là một cây nhị phân dùng để chia nhỏ dữ liệu bằng các phép chia ngẫu nhiên. Mục tiêu của cây không phải là học nhãn hay tối ưu một hàm mất mát nào đó, mà là tạo ra một quá trình chia dữ liệu để quan sát xem điểm nào dễ bị tách riêng khỏi phần còn lại.




Hình 4: **Minh họa cách Isolation Tree tách điểm bất thường.** Cây chia dữ liệu nhiều lần; điểm nằm xa nhóm chính thường bị tách ra sớm hơn các điểm bình thường.

Cụ thể, tại mỗi node của *iTree*, thuật toán chọn ngẫu nhiên một feature, sau đó chọn ngẫu nhiên một giá trị split nằm giữa giá trị nhỏ nhất và lớn nhất của feature đó trong node hiện tại. Dựa trên giá trị split này, dữ liệu tại node được chia thành hai nhánh: các điểm có giá trị nhỏ hơn split value đi sang một nhánh, còn các điểm có giá trị lớn hơn hoặc bằng split value đi sang nhánh còn lại.

Quá trình chia tiếp tục được thực hiện đệ quy trên các node con. Cây sẽ dừng phát triển khi node chỉ còn một điểm dữ liệu, khi các điểm trong node có cùng giá trị, hoặc khi cây đạt đến độ sâu giới hạn. Khi một điểm dữ liệu rơi vào node chỉ chứa chính nó, ta nói điểm đó đã được cô lập.

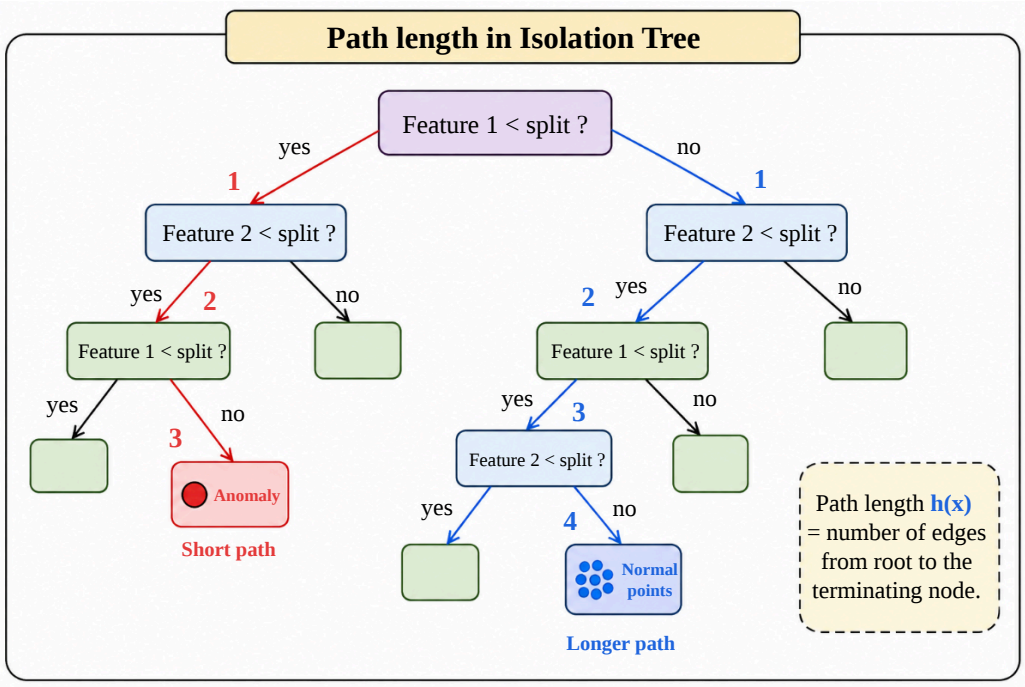
Điểm quan trọng là *iTree* không cố gắng tìm cách chia "tốt nhất". Điều này làm nó khác với decision tree trong bài toán classification. Decision tree thường tìm split sao cho việc dự đoán nhãn tốt hơn, ví dụ dựa trên Gini, entropy hoặc classification loss. Ngược lại, *iTree* chỉ dùng các phép chia ngẫu nhiên để đo độ dễ bị cô lập của từng điểm.

**Lưu ý**

*iTree* không phải là một decision tree dùng để phân loại. Nó là một cây ngẫu nhiên dùng để đo xem một điểm dữ liệu cần bao nhiêu lần chia để được tách riêng khỏi phần còn lại.

## 2. Path Length

Đối với một điểm dữ liệu $x$, ta cho điểm đó đi từ root của cây xuống đến node nơi nó được cô lập. Số bước chia mà $x$ đi qua được gọi là *path length*, ký hiệu là $h(x)$.




Hình 5: **Minh họa path length trong Isolation Tree.** Điểm bất thường thường có đường đi ngắn hơn vì nó bị tách ra khỏi dữ liệu sớm hơn điểm bình thường.

Path length là đại lượng cốt lõi trong Isolation Forest. Nếu một điểm có path length ngắn, nghĩa là điểm đó bị tách riêng rất nhanh. Điều này cho thấy điểm đó có thể nằm ở vùng thưa dữ liệu hoặc khác biệt rõ so với phần lớn dữ liệu. Ngược lại, nếu một điểm có path length dài, nghĩa là nó khó bị cô lập hơn và thường nằm trong vùng có nhiều điểm tương tự xung quanh.

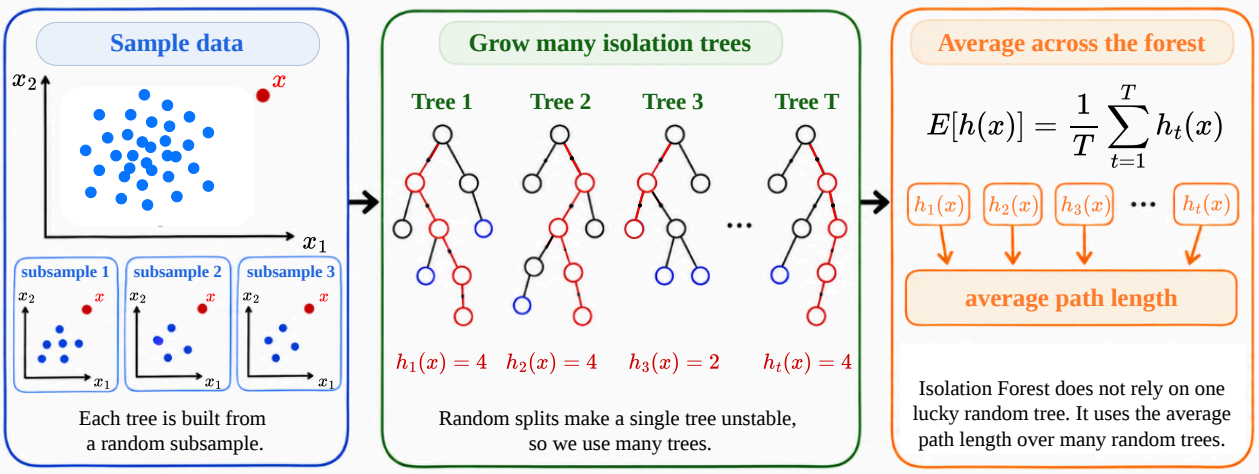
**Điểm cần nhớ**

- $h(x)$ nhỏ $\Rightarrow$ điểm dễ bị cô lập $\Rightarrow$ có xu hướng là anomaly.
- $h(x)$ lớn $\Rightarrow$ điểm khó bị cô lập $\Rightarrow$ có xu hướng là normal.

Nói cách khác, Isolation Forest không đo trực tiếp khoảng cách đến tâm cụm hay mật độ lân cận. Thay vào đó, nó đo xem một điểm cần bao nhiêu lần chia ngẫu nhiên để bị tách ra khỏi phần còn lại.

## 3. Từ một cây đến một Forest

Một Isolation Tree đơn lẻ có thể cho kết quả không ổn định. Vì cây được xây bằng các phép chia ngẫu nhiên, một lần xây cây có thể vô tình cô lập anomaly rất nhanh, nhưng một lần khác lại chia không thuận lợi và khiến cùng điểm đó đi sâu hơn trong cây.




Hình 6: **Minh họa cách Isolation Forest hoạt động.** Thuật toán xây nhiều cây ngẫu nhiên và dùng độ dài đường đi trung bình để đánh giá một điểm có bất thường hay không.

Để giảm sự phụ thuộc vào một cây ngẫu nhiên duy nhất, Isolation Forest xây dựng nhiều Isolation Trees. Mỗi cây thường được xây trên một mẫu con của dữ liệu. Sau đó, với mỗi điểm $x$, ta tính path length của nó trên từng cây và lấy trung bình:

$$\mathbb{E}[h(x)] = \frac{1}{T}\sum_{t=1}^{T} h_t(x),$$

trong đó:

- $T$ là số lượng cây trong forest;
- $h_t(x)$ là path length của điểm $x$ trên cây thứ $t$;
- $\mathbb{E}[h(x)]$ là path length trung bình của điểm $x$.

Việc lấy trung bình qua nhiều cây giúp kết quả ổn định hơn. Nếu một điểm thực sự dễ bị cô lập, nó thường sẽ có path length ngắn trên nhiều cây khác nhau. Ngược lại, một điểm bình thường thường cần nhiều bước hơn để bị cô lập trên phần lớn các cây.

**Ý nghĩa của forest**

Isolation Forest dùng nhiều cây ngẫu nhiên để giảm nhiễu từ từng cây đơn lẻ. Kết quả cuối cùng không dựa vào một lần chia may rủi, mà dựa vào xu hướng trung bình qua nhiều cây.

## 4. Anomaly Score

Sau khi tính được path length trung bình, Isolation Forest cần chuyển đại lượng này thành một điểm số để đánh giá mức độ bất thường của từng mẫu. Điểm số này được gọi là *anomaly score*. Mục tiêu của anomaly score là rất trực quan: điểm nào bị cô lập càng nhanh thì điểm đó càng đáng nghi.

Anomaly score của một điểm dữ liệu $x$ được định nghĩa như sau:

$$s(x, \psi) = 2^{-\frac{\mathbb{E}[h(x)]}{c(\psi)}}.$$

Trong công thức này, $\mathbb{E}[h(x)]$ là path length trung bình của điểm $x$ qua nhiều isolation trees. Giá trị $\psi$ là kích thước mẫu con dùng để xây mỗi cây, còn $c(\psi)$ là hệ số chuẩn hóa theo kích thước mẫu.

Hệ số chuẩn hóa $c(\psi)$ được dùng để so sánh path length trong các cây được xây từ các mẫu con có cùng kích thước. Nó được định nghĩa như sau:

$$c(\psi) = \begin{cases} 2H_{\psi-1} - \dfrac{2(\psi-1)}{\psi}, & \psi > 2, \\ 1, & \psi = 2, \\ 0, & \psi \leq 1. \end{cases}$$

Trong đó $H_{\psi-1}$ là harmonic number:

$$H_{\psi-1} = \sum_{i=1}^{\psi-1} \frac{1}{i}.$$

Tuy công thức có vẻ hơi phức tạp, ý nghĩa chính của nó khá đơn giản. Nếu một điểm có $\mathbb{E}[h(x)]$ nhỏ, tức là điểm đó thường bị cô lập sau rất ít lần chia, thì giá trị $s(x, \psi)$ sẽ lớn. Ngược lại, nếu một điểm có $\mathbb{E}[h(x)]$ lớn, tức là điểm đó cần nhiều lần chia mới bị cô lập, thì giá trị $s(x, \psi)$ sẽ nhỏ hơn.

Có thể hiểu anomaly score như sau:

- $s(x, \psi)$ gần 1: điểm rất dễ bị cô lập, nên rất đáng nghi;
- $s(x, \psi)$ quanh 0.5: điểm nằm ở vùng chưa thật sự rõ ràng;
- $s(x, \psi)$ nhỏ hơn 0.5: điểm có xu hướng bình thường hơn.

**Điểm cần nhớ**

Isolation Forest không trực tiếp học ranh giới giữa normal và anomaly. Thay vào đó, thuật toán đo xem mỗi điểm cần bao nhiêu lần chia ngẫu nhiên để được cô lập. Path length càng ngắn thì anomaly score càng cao, và điểm đó càng đáng nghi.

# IV. Thực hành

## 1. Bộ dữ liệu

Trong phần thực hành, ta sử dụng **ODDS Shuttle dataset**. ODDS là một bộ dữ liệu thường được dùng cho các bài toán outlier detection và anomaly detection. Shuttle là dataset dạng bảng, gồm các đặc trưng số và có sẵn nhãn anomaly.

Bảng 1: Thông tin tổng quan về bộ dữ liệu Shuttle.

| Thuộc tính | Ý nghĩa |
|---|---|
| Tên dataset | Shuttle |
| Nguồn | ODDS: Outlier Detection DataSets |
| Dạng dữ liệu | Tabular / numerical |
| Số chiều đặc trưng | 9 |
| Bài toán | Outlier detection / anomaly detection |
| Nhãn | Có nhãn inlier/outlier |

Trong ví dụ này, dữ liệu được lưu dưới dạng file `shuttle.mat`. File này chứa ma trận đặc trưng `X` và nhãn `y`. Mặc dù Isolation Forest có thể huấn luyện không cần nhãn, ta vẫn dùng nhãn của Shuttle để đánh giá mô hình sau khi dự đoán.

**Lưu ý**

Việc dataset có nhãn nhưng Isolation Forest vẫn được huấn luyện theo kiểu unsupervised learning. Mô hình vẫn học theo hướng unsupervised. Nhãn `y` chỉ được dùng ở bước đánh giá kết quả.

## 2. Output trong Scikit-learn

Trong Scikit-learn, ta sử dụng class `IsolationForest` như sau:

```
from sklearn.ensemble import IsolationForest
```

Sau khi huấn luyện mô hình, có hai output quan trọng cần hiểu rõ:

```
predictions = model.predict(X)
scores = model.decision_function(X)
```

Hàm `predict(X)` trả về nhãn theo quy ước của Scikit-learn:

Bảng 2: Giá trị nhãn của mô hình phát hiện bất thường.

| Giá trị | Ý nghĩa |
|---|---|
| `1` | Mẫu bình thường |
| `-1` | Mẫu bất thường |

Với `decision_function(X)`, giá trị càng thấp thì mẫu càng đáng nghi. Để kết quả dễ đọc hơn, trong code ta đổi dấu score:

```
anomaly_score = -model.decision_function(X)
```

Sau khi đổi dấu, ta có quy ước:

Bảng 3: `anomaly_score` sau khi đổi dấu từ `decision_function`.

| Giá trị | Ý nghĩa |
|---|---|
| `anomaly_score` cao | Mẫu có khả năng bất thường cao hơn, cần được chú ý hơn. |
| `anomaly_score` thấp | Mẫu có xu hướng bình thường hơn, ít đáng nghi hơn. |

**Lưu ý**

Trong nhiều bài toán classification, ta thường dùng `1` để biểu diễn lớp positive hoặc anomaly. Nhưng với `sklearn.ensemble.IsolationForest`, nhãn anomaly là `-1`, còn nhãn normal là `1`.

## 3. Pipeline tổng quát

Pipeline thực hành gồm các bước sau:




Hình 7: Pipeline tổng quát.

Trong pipeline này, bước preprocessing giúp dữ liệu đầu vào nhất quán trước khi đưa vào mô hình. Sau đó, Isolation Forest được huấn luyện trên dữ liệu đã xử lý. Mô hình trả về nhãn dự đoán và anomaly score. Cuối cùng, vì Shuttle có nhãn thật, ta có thể đánh giá mô hình bằng các metric như precision, recall, F1-score và ROC-AUC.

## 4. Cài đặt chương trình

Trong phần cài đặt, ta sử dụng trực tiếp class `IsolationForest` có sẵn trong thư viện `scikit-learn`, thay vì tự triển khai thuật toán từ đầu. Notebook Google Colab được dùng để tải dữ liệu, huấn luyện mô hình, dự đoán anomaly và đánh giá kết quả.

### 4.1 Cài đặt và import thư viện

```
!pip -q install scipy scikit-learn pandas matplotlib gdown
```

```
from pathlib import Path

import gdown
import numpy as np
import pandas as pd
import matplotlib.pyplot as plt

from scipy.io import loadmat
from IPython.display import HTML, display

from sklearn.ensemble import IsolationForest
from sklearn.metrics import classification_report, confusion_matrix, roc_auc_score
from sklearn.preprocessing import StandardScaler
```

Phần import quan trọng nhất là `IsolationForest` từ `sklearn.ensemble`. Đây là class có sẵn của Scikit-learn để xây dựng mô hình Isolation Forest. Ngoài ra, `StandardScaler` được dùng để chuẩn hóa dữ liệu trước khi đưa vào mô hình.

### 4.2 Tải và đọc dữ liệu

Dataset `shuttle.mat` được tải từ Google Drive và đọc bằng hàm `loadmat`. Trong file này, `X` là ma trận đặc trưng và `y` là nhãn thật dùng cho bước đánh giá.

```
DRIVE_FILE_ID = "___"
DATA_PATH = "data/shuttle.mat"
Path("data").mkdir(parents=True, exist_ok=True)

!gdown --id {DRIVE_FILE_ID} -O {DATA_PATH}

data = loadmat(DATA_PATH)

X = pd.DataFrame(data["X"])
y = pd.Series(data["y"].ravel()).astype(int)
print("X shape:", X.shape)
print(y.value_counts().sort_index())
```

Trong đoạn code trên, `DRIVE_FILE_ID` là mã định danh của file dữ liệu trên Google Drive. Biến `DATA_PATH` là đường dẫn lưu file sau khi tải về Colab. Lệnh `Path("data").mkdir(...)` tạo thư mục `data` nếu thư mục này chưa tồn tại.

Sau khi tải xong, hàm `loadmat` đọc nội dung file `.mat`. Dữ liệu đặc trưng được lấy từ khóa `X`, còn nhãn thật được lấy từ khóa `y`. Dòng `ravel()` chuyển nhãn về dạng vector một chiều, giúp dễ dùng với các hàm đánh giá của Scikit-learn.

### 4.3 Chuẩn hóa dữ liệu

Trước khi huấn luyện, dữ liệu được chuẩn hóa bằng `StandardScaler`.

```
scaler = StandardScaler()
X_scaled = scaler.fit_transform(X)
```

`StandardScaler` biến đổi mỗi feature sao cho có trung bình gần 0 và độ lệch chuẩn gần 1. Bước này giúp các feature có thang đo khác nhau trở nên cân bằng hơn trước khi đưa vào mô hình. Biến `X_scaled` là dữ liệu sau khi đã được chuẩn hóa.

### 4.4 Huấn luyện Isolation Forest

Mô hình được tạo trực tiếp từ `sklearn.ensemble.IsolationForest`. Sau khi huấn luyện, hàm `predict` trả về `1` cho mẫu bình thường và `-1` cho anomaly.

```
model = IsolationForest(
    n_estimators=300,
    max_samples="auto",
    contamination="auto",
    random_state=42,
    n_jobs=-1,
)

model.fit(X_scaled)

predictions = model.predict(X_scaled)
scores = model.decision_function(X_scaled)
```

Các tham số chính trong mô hình gồm:

- `n_estimators=300`: số lượng isolation trees trong forest. Số cây càng nhiều thì kết quả thường ổn định hơn, nhưng thời gian huấn luyện cũng tăng lên.
- `max_samples="auto"`: số lượng mẫu được dùng để xây mỗi cây. Khi đặt là `"auto"`, `scikit-learn` tự chọn kích thước mẫu phù hợp theo thiết lập mặc định.
- `contamination="auto"`: tỉ lệ anomaly giả định trong dữ liệu. Khi chưa biết trước tỉ lệ anomaly, ta có thể dùng `"auto"` để Scikit-learn tự xác định ngưỡng quyết định.
- `random_state=42`: cố định seed ngẫu nhiên để kết quả có thể tái lập khi chạy lại notebook.

- `n_jobs=-1`: sử dụng tất cả CPU cores có sẵn để tăng tốc quá trình huấn luyện và dự đoán.

Sau khi gọi `fit`, mô hình được huấn luyện trên dữ liệu `X_scaled`. Hàm `predict` trả về nhãn dự đoán theo quy ước của Scikit-learn, còn `decision_function` trả về điểm số cho biết mức độ bình thường hoặc bất thường của từng mẫu.

### 4.5 Tạo anomaly score

Scikit-learn quy ước `decision_function` càng thấp thì mẫu càng bất thường. Vì vậy, ta đổi dấu score để `anomaly_score` càng cao thì mẫu càng đáng nghi.

```
is_anomaly = (predictions == -1).astype(int)
anomaly_score = -scores

results = X.copy()
results["sklearn_label"] = predictions
results["is_anomaly"] = is_anomaly
results["anomaly_score"] = anomaly_score

results = results.sort_values("anomaly_score", ascending=False)

display(results.head(20))
```

Biến `is_anomaly` chuyển nhãn của Scikit-learn sang dạng dễ dùng hơn: `0` là normal và `1` là anomaly. Biến `anomaly_score` được tạo bằng cách đổi dấu `scores`, để giá trị càng lớn thì mẫu càng đáng nghi.

Bảng `results` chứa dữ liệu gốc, nhãn dự đoán từ Scikit-learn, nhãn anomaly dạng nhị phân và anomaly score. Sau đó, bảng được sắp xếp giảm dần theo `anomaly_score` để các mẫu đáng nghi nhất xuất hiện ở đầu bảng.

## 5. Đánh giá mô hình

Isolation Forest là mô hình unsupervised, nên nhãn `y` không được dùng khi huấn luyện. Nhãn thật chỉ được dùng sau cùng để đánh giá chất lượng phát hiện anomaly.

```
print("Confusion matrix:")
print(confusion_matrix(y, is_anomaly))

print("\nClassification report:")
print(classification_report(y, is_anomaly, digits=4))

print(f"\nROC-AUC: {roc_auc_score(y, anomaly_score):.4f}")
```

Trong phần đánh giá, `confusion_matrix` cho biết số lượng mẫu normal và anomaly được dự đoán đúng hoặc sai. Hàm `classification_report` in ra các metric như precision, recall và

F1-score. Trong đó, precision cho biết trong các mẫu được dự đoán là anomaly thì có bao nhiêu mẫu thật sự là anomaly, còn recall cho biết mô hình tìm được bao nhiêu anomaly trong toàn bộ anomaly thật.

Cuối cùng, `roc_auc_score` đánh giá khả năng xếp hạng anomaly của mô hình dựa trên `anomaly_score`. Giá trị ROC-AUC càng cao thì mô hình càng có khả năng xếp anomaly cao hơn normal.

**Lưu ý**

Trong anomaly detection, không nên chỉ dựa vào accuracy vì dữ liệu thường mất cân bằng mạnh. Một mô hình luôn dự đoán tất cả là normal vẫn có thể đạt accuracy cao, nhưng mô hình đó không phát hiện được anomaly.

# V. Tài liệu tham khảo

[1] F. T. Liu et al., "Isolation forest", in *2008 Eighth IEEE International Conference on Data Mining*, IEEE, 2008, pp. 413–422. DOI: 10.1109/ICDM.2008.17. [Online]. Available: https://doi.org/10.1109/ICDM.2008.17.

[2] Scikit-learn Developers, *Sklearn.ensemble.isolationforest*, Scikit-learn Documentation, Accessed: 2026-06-02, 2026. [Online]. Available: https://scikit-learn.org/stable/modules/generated/sklearn.ensemble.IsolationForest.html.

[3] S. Rayana, *Shuttle dataset*, ODDS: Outlier Detection DataSets, Accessed: 2026-06-02, 2026. [Online]. Available: https://shebuti.com/shuttle-dataset/.

# Phụ lục

1. **Source:** Các file dataset và code trong bài có thể được tải tại đây.




Hình 8: Hình ảnh group facebook AIO Q&A.